

Question 1: Real-time Notifications with Django Channels + React

You are building a school management system. When a new student registers, the admin should get a **real-time notification** in the React dashboard using **Django Channels** and WebSockets.

Input Format:

- Student Registration API request

```
{  
  "name": "Rahul Sharma",  
  "class": "10",  
  "section": "B"  
}
```

Output Format (React WebSocket Message):

```
{  
  "type": "NEW_REGISTRATION",  
  "message": "New student Rahul Sharma registered in Class 10-B"  
}
```

Example 1

Input:

```
{  
  "name": "Anjali Mehta",  
  "class": "12",  
  "section": "A"  
}
```

Output:

```
{  
  "type": "NEW_REGISTRATION",  
  "message": "New student Anjali Mehta registered in Class 12-A"  
}
```

Example 2

Input:

```
{
  "name": "Vikram Singh",
  "class": "8",
  "section": "C"
}
```

Output:

```
{
  "type": "NEW_REGISTRATION",
  "message": "New student Vikram Singh registered in Class 8-C"
}
```

Answer :

models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    class_name = models.CharField(max_length=10)
    section = models.CharField(max_length=5)
    def __str__(self):
        return f'{self.name} ({self.class_name}-{self.section})'
```

views.py

```
from rest_framework.response import Response
from rest_framework.decorators import api_view
from .models import Student
from .serializers import StudentSerializer
from channels.layers import get_channel_layer
from asgiref.sync import async_to_sync

@api_view(['POST'])
def register_student(request):
```

```

serializer = StudentSerializer(data=request.data)
if serializer.is_valid():
    student = serializer.save()
    channel_layer = get_channel_layer()
    async_to_sync(channel_layer.group_send)(
        "admins",
        {
            "type": "send_notification",
            "message": f"New student {student.name} registered in Class
{student.class_name}-{student.section}"
        }
    )
    return Response(serializer.data)
return Response(serializer.errors, status=400)

```

serializers.py

```

from rest_framework import serializers
from .models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = ['id', 'name', 'class_name', 'section']

```

consumers.py

```

import json

from channels.generic.websocket import AsyncWebsocketConsumer

class AdminConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        await self.channel_layer.group_add("admins", self.channel_name)

```

```

        await self.accept()
    async def disconnect(self, close_code):
        await self.channel_layer.group_discard("admins", self.channel_name)
    async def send_notification(self, event):
        await self.send(text_data=json.dumps({
            "type": "NEW_REGISTRATION",
            "message": event["message"]
        }))

```

routing.py

```

from django.urls import path
from .consumers import AdminConsumer
websocket_urlpatterns = [
    path("ws/notifications/", AdminConsumer.as_asgi()),
]

```

asgi.py

```

import os

from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
import app_name.routing

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'project_name.settings')

application = ProtocolTypeRouter({
    "http": get_asgi_application(),
    "websocket": AuthMiddlewareStack(
        URLRouter(
            app_name.routing.websocket_urlpatterns
        )
    )
})

```

```
),  
})
```

useNotifications.js

```
import { useEffect, useState } from "react";  
export default function useNotifications() {  
  const [notifications, setNotifications] = useState([]);  
  useEffect(() => {  
    const socket = new WebSocket("ws://localhost:8000/ws/notifications/");  
    socket.onmessage = (event) => {  
      const data = JSON.parse(event.data);  
      setNotifications((prev) => [...prev, data.message]);  
    };  
    return () => socket.close();  
  }, []);  
  return notifications;  
}
```

AdminDashboard.js

```
import React from "react";  
import useNotifications from "../useNotifications";  
function AdminDashboard() {  
  const notifications = useNotifications();  
  return (  
    <div>  
      <h2>Admin Dashboard</h2>  
      <h3>Notifications:</h3>  
      <ul>  
        {notifications.map((msg, idx) => (  
          <li>{msg}</li>  
        ))}
```

```

        <li key={idx}>{msg}</li>
      )}}
    </ul>
  </div>
);
}
export default AdminDashboard;

```

Question 2: File Upload + Background Processing (Celery + React Progress Bar)

A teacher uploads a huge Excel file of student marks. The backend should process it in the background using **Celery**, and React should show **progress updates** via an API.

Input Format:

- Teacher uploads file → API receives multipart/form-data

POST /api/upload-marks

Content-Type: multipart/form-data

file=marks.xlsx

Output Format (Progress API Response):

```

{
  "task_id": "12345",
  "status": "IN_PROGRESS",
  "progress": "45%"
}

```

Example 1

Input: Teacher uploads marks_class10.xlsx

Output:

```

{
  "task_id": "98765",
  "status": "QUEUED",
  "progress": "0%"
}

```

```
}
```

Example 2

Input: API call to check status

GET /api/task-status/98765

Output:

```
{  
    "task_id": "98765",  
    "status": "COMPLETED",  
    "progress": "100%",  
    "message": "Marks successfully uploaded for 80 students"  
}
```

Answer :

tasks.py

```
import time  
  
from celery import shared_task  
from celery_progress.backend import ProgressRecorder  
  
@shared_task(bind=True)  
def process_marks(self, file_path):  
    progress_recorder = ProgressRecorder(self)  
    total = 100 # suppose 100 students for demo  
    for i in range(1, total + 1):  
        time.sleep(0.1) # simulate work  
        progress_recorder.set_progress(i, total, f"Processing student {i}")  
    return f"Marks successfully uploaded for {total} students"
```

views.py

```
import os  
  
from django.conf import settings  
from rest_framework.response import Response
```

```

from rest_framework.decorators import api_view
from .tasks import process_marks
from celery.result import AsyncResult

@api_view(['POST'])
def upload_marks(request):
    file = request.FILES['file']
    file_path = os.path.join(settings.MEDIA_ROOT, file.name)
    with open(file_path, 'wb+') as destination:
        for chunk in file.chunks():
            destination.write(chunk)
    task = process_marks.delay(file_path)
    return Response({
        "task_id": task.id,
        "status": "QUEUED",
        "progress": "0%"
    })

@api_view(['GET'])
def task_status(request, task_id):
    result = AsyncResult(task_id)
    response = {"task_id": task_id, "status": result.status}
    if result.status == "PROGRESS":
        response["progress"] = result.info.get("current", 0) * 100
        response["progress"] = f"{response['progress']}%"
    elif result.status == "SUCCESS":
        response["progress"] = "100%"
        response["message"] = result.result
    else:
        response["progress"] = "0%"

```

```
return Response(response)
```

urls.py

```
from django.urls import path
from .views import upload_marks, task_status
urlpatterns = [
    path("upload-marks/", upload_marks, name="upload_marks"),
    path("task-status/<str:task_id>/", task_status, name="task_status"),
]
```

celery.py

```
import os
from celery import Celery
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "project_name.settings")
app = Celery("project_name")
app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()
```

UploadMarks.js

```
import React, { useState } from "react";
import axios from "axios";
function UploadMarks() {
    const [taskId, setTaskId] = useState(null);
    const [progress, setProgress] = useState("0%");
    const [status, setStatus] = useState("");
    const handleUpload = async (e) => {
        e.preventDefault();
        const file = e.target.file.files[0];
        const formData = new FormData();
```

```

formData.append("file", file);

const res = await axios.post("/api/upload-marks/", formData, {
  headers: { "Content-Type": "multipart/form-data" }
});

setTaskId(res.data.task_id);
setStatus(res.data.status);

const interval = setInterval(async () => {
  const statusRes = await axios.get(`/api/task-status/${res.data.task_id}/`);
  setProgress(statusRes.data.progress);
  setStatus(statusRes.data.status);
  if (statusRes.data.status === "COMPLETED") {
    clearInterval(interval);
    alert(statusRes.data.message);
  }
}, 1000);
};

return (
  <div>
    <h2>Upload Marks</h2>
    <form onSubmit={handleUpload}>
      <input type="file" name="file" required />
      <button type="submit">Upload</button>
    </form>
    {taskId && (
      <div>
        <p>Status: {status}</p>
        <progress value={parseInt(progress)} max="100"></progress>
        <span>{progress}</span>
      </div>
    )}
  </div>

```

```

    })
  </div>

);
}

export default UploadMarks;

```

Question 3: JWT Authentication with Refresh Token (Django + React)

Implement authentication where the React app requests a **new access token** when the old one expires using a **refresh token API**.

Input Format:

```

{
  "refresh": "eyJ0eXAiOiJKV1QiLCJh..."
}

```

Output Format:

```

{
  "access": "eyJhbGciOiJIUzI1NiIs..."
}

```

Example 1

Input:

```

{
  "refresh": "refresh_token_123"
}

```

Output:

```

{
  "access": "access_token_987"
}

```

Example 2

Input: Expired refresh token

```

{
  "refresh": "expired_token"
}

```

```
}
```

Output:

```
{
```

```
  "error": "Invalid refresh token"
```

```
}
```

Answer :

urls.py

```
from django.urls import path

from rest_framework_simplejwt.views import TokenObtainPairView,
TokenRefreshView

urlpatterns = [

    path("api/token/", TokenObtainPairView.as_view(), name="token_obtain_pair"),
    path("api/token/refresh/", TokenRefreshView.as_view(), name="token_refresh"),
]
```

api.js

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:8000/api/",
});

api.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response.status === 401) {
      const refresh = localStorage.getItem("refresh");
      if (refresh) {
        try {
          const res = await axios.post("http://localhost:8000/api/token/refresh/",
{ refresh });

          localStorage.setItem("access", res.data.access);
```

```

        error.config.headers["Authorization"] = `Bearer ${res.data.access}`;
        return axios(error.config); // retry original request
    } catch {
        localStorage.clear();
        window.location.href = "/login";
    }
}
}
}
return Promise.reject(error);
}
);
export default api;

```

Login.js

```

import React, { useState } from "react";
import axios from "axios";

function Login() {
    const [username, setUsername] = useState("");
    const [password, setPassword] = useState("");
    const handleLogin = async (e) => {
        e.preventDefault();
        const res = await axios.post("http://localhost:8000/api/token/", {
            username,
            password,
        });
        localStorage.setItem("access", res.data.access);
        localStorage.setItem("refresh", res.data.refresh);
        alert("Login successful!");
    };
};

```

```

return (
  <form onSubmit={handleLogin}>
    <input value={username} onChange={(e) => setUsername(e.target.value)}
placeholder="Username" />
    <input type="password" value={password} onChange={(e) =>
setPassword(e.target.value)} placeholder="Password" />
    <button type="submit">Login</button>
  </form>
);
}
export default Login;

```

Question 4: Complex Filtering API with React Search Bar

Create an API in Django that supports **multi-parameter filtering** of students (name, class, section, attendance > 80%). React should call it via search.

Input Format (Query Params):

GET /api/students?class=10§ion=A&attendance__gte=80

Output Format:

```
[
  {"id": 1, "name": "Rahul Sharma", "class": "10", "section": "A", "attendance": 92},
  {"id": 2, "name": "Priya Kapoor", "class": "10", "section": "A", "attendance": 85}
]
```

Example 1

Input:

GET /api/students?class=12&attendance__gte=90

Output:

```
[
  {"id": 5, "name": "Rohit Mehta", "class": "12", "section": "B", "attendance": 95}
]
```

Example 2

Input:

GET /api/students?name=Anjali

Output:

```
[  
  {"id": 7, "name": "Anjali Mehta", "class": "11", "section": "C", "attendance": 88}  
]
```

Answer :

models.py

```
from django.db import models  
  
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    class_name = models.CharField(max_length=10)  
    section = models.CharField(max_length=5)  
    attendance = models.IntegerField()  
    def __str__(self):  
        return f'{self.name} ({self.class_name}-{self.section})'
```

serializers.py

```
from rest_framework import serializers  
from .models import Student  
  
class StudentSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Student  
        fields = ['id', 'name', 'class_name', 'section', 'attendance']
```

views.py

```
from rest_framework import generics  
from django_filters.rest_framework import DjangoFilterBackend  
from .models import Student  
from .serializers import StudentSerializer
```

```

class StudentListView(generics.ListAPIView):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
    filter_backends = [DjangoFilterBackend]
    filterset_fields = {
        'name': ['icontains'],
        'class_name': ['exact'],
        'section': ['exact'],
        'attendance': ['gte', 'lte'] # >=, <= filtering
    }

```

urls.py

```

from django.urls import path
from .views import StudentListView
urlpatterns = [
    path("students/", StudentListView.as_view(), name="student_list"),
]

```

StudentSearch.js

```

import React, { useState } from "react";
import axios from "axios";

function StudentSearch() {
    const [filters, setFilters] = useState({ name: "", class_name: "", section: "",
attendance: "" });

    const [students, setStudents] = useState([]);

    const handleChange = (e) => {
        setFilters({ ...filters, [e.target.name]: e.target.value });
    };
}

```

```

const handleSearch = async () => {
  const params = {};
  if (filters.name) params.name__icontains = filters.name;
  if (filters.class_name) params.class_name = filters.class_name;
  if (filters.section) params.section = filters.section;
  if (filters.attendance) params.attendance__gte = filters.attendance;
  const res = await axios.get("http://localhost:8000/api/students/", { params });
  setStudents(res.data);
};

return (
  <div>
    <h2>Search Students</h2>
    <input name="name" placeholder="Name" onChange={handleChange} />
    <input name="class_name" placeholder="Class" onChange={handleChange} />
    <input name="section" placeholder="Section" onChange={handleChange} />
    <input name="attendance" placeholder="Min Attendance"
    onChange={handleChange} />
    <button onClick={handleSearch}>Search</button>
    <ul>
      {students.map(s => (
        <li key={s.id}>
          {s.name} - Class {s.class_name}-{s.section} (Attendance: {s.attendance}%)
        </li>
      ))}
    </ul>
  </div>
);
}

```

```
export default StudentSearch;
```

Question 5: Role-Based Access Control (RBAC) in Django + React

Implement RBAC where only teachers can update marks and students can only view them.

Input Format (Update Marks Request by Teacher):

```
{  
  "user_role": "teacher",  
  "student_id": 101,  
  "marks": 88  
}
```

Output Format:

```
{  
  "status": "success",  
  "message": "Marks updated for student 101"  
}
```

Example 1

Input:

```
{  
  "user_role": "teacher",  
  "student_id": 102,  
  "marks": 92  
}
```

Output:

```
{  
  "status": "success",  
  "message": "Marks updated for student 102"  
}
```

Example 2

Input (Student trying to update):

```
{
  "user_role": "student",
  "student_id": 103,
  "marks": 95
}
```

Output:

```
{
  "status": "error",
  "message": "Permission denied. Only teachers can update marks."
}
```

Answer:

models.py

```
from django.db import models
from django.contrib.auth.models import AbstractUser
class User(AbstractUser):
    ROLE_CHOICES = (
        ("teacher", "Teacher"),
        ("student", "Student"),
    )
    role = models.CharField(max_length=10, choices=ROLE_CHOICES,
default="student")
class Student(models.Model):
    name = models.CharField(max_length=100)
    marks = models.IntegerField(default=0)
```

serializers.py

```
from rest_framework import serializers
from .models import Student
```

```
class StudentSerializer(serializers.ModelSerializer):
```

```
    class Meta:
```

```
        model = Student
```

```
        fields = ['id', 'name', 'marks']
```

views.py

```
from rest_framework.response import Response
```

```
from rest_framework.decorators import api_view, permission_classes
```

```
from rest_framework.permissions import IsAuthenticated
```

```
from .models import Student
```

```
from .serializers import StudentSerializer
```

```
@api_view(['POST'])
```

```
@permission_classes([IsAuthenticated])
```

```
def update_marks(request):
```

```
    user = request.user
```

```
    if user.role != "teacher":
```

```
        return Response({
```

```
            "status": "error",
```

```
            "message": "Permission denied. Only teachers can update marks."
```

```
        }, status=403)
```

```
    student_id = request.data.get("student_id")
```

```
    marks = request.data.get("marks")
```

```
    try:
```

```
        student = Student.objects.get(id=student_id)
```

```
        student.marks = marks
```

```
        student.save()
```

```
    return Response({
```

```
        "status": "success",
```

```
        "message": f"Marks updated for student {student_id}"
```

```

    })

    except Student.DoesNotExist:

        return Response({"status": "error", "message": "Student not found"},
            status=404)

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def view_marks(request):

    students = Student.objects.all()

    serializer = StudentSerializer(students, many=True)

    return Response(serializer.data)

```

urls.py

```

from django.urls import path

from .views import update_marks, view_marks

urlpatterns = [

    path("students/view/", view_marks, name="view_marks"),

    path("students/update/", update_marks, name="update_marks"),

]

```

MarksDashboard.js

```

import React, { useState, useEffect } from "react";
import axios from "axios";

function MarksDashboard({ userRole }) {

    const [students, setStudents] = useState([]);

    useEffect(() => {

        axios.get("http://localhost:8000/api/students/view/").then((res) => {

            setStudents(res.data);

        });

    }, []);
}

```

```

const updateMarks = async (id, newMarks) => {
  const res = await axios.post("http://localhost:8000/api/students/update/", {
    student_id: id,
    marks: newMarks,
    user_role: userRole,
  });
  alert(res.data.message);
};

return (
  <div>
    <h2>Student Marks</h2>
    <ul>
      {students.map((s) => (
        <li key={s.id}>
          {s.name} - Marks: {s.marks}
          {userRole === "teacher" && (
            <button onClick={() => updateMarks(s.id, s.marks + 5)}>+5 Marks</button>
          )}
        </li>
      ))}
    </ul>
  </div>
);
}

export default MarksDashboard;

```

Question 6: Pagination + Infinite Scroll in React with Django API

Django API must return paginated results, and React should load more students on scroll.

Input Format:

GET /api/students?page=2&page_size=3

Output Format:

```
{
  "page": 2,
  "results": [
    {"id": 4, "name": "Neha Gupta"},
    {"id": 5, "name": "Amit Kumar"},
    {"id": 6, "name": "Sneha Reddy"}
  ]
}
```

Example 1**Input:**

GET /api/students?page=1&page_size=2

Output:

```
{
  "page": 1,
  "results": [
    {"id": 1, "name": "Rahul Sharma"},
    {"id": 2, "name": "Anjali Mehta"}
  ]
}
```

Example 2**Input:**

GET /api/students?page=3&page_size=2

Output:

```
{
  "page": 3,
  "results": [
    {"id": 5, "name": "Amit Kumar"},
  ]
}
```

```
    {"id": 6, "name": "Sneha Reddy"}  
]  
}
```

Answer :

models.py

```
from django.db import models  
  
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    def __str__(self):  
        return self.name
```

serializers.py

```
from rest_framework import serializers  
from .models import Student  
  
class StudentSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Student  
        fields = ['id', 'name']
```

pagination.py

```
from rest_framework.pagination import PageNumberPagination  
from rest_framework.response import Response  
  
class StudentPagination(PageNumberPagination):  
    page_size_query_param = 'page_size'  
    def get_paginated_response(self, data):  
        return Response({  
            "page": self.page.number,  
            "results": data  
        })
```

views.py

```
from rest_framework import generics
from .models import Student
from .serializers import StudentSerializer
from .pagination import StudentPagination
class StudentListView(generics.ListAPIView):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
    pagination_class = StudentPagination
```

urls.py

```
from django.urls import path
from .views import StudentListView
urlpatterns = [
    path("students/", StudentListView.as_view(), name="student_list"),
]
```

StudentList.js

```
import React, { useState, useEffect } from "react";
import axios from "axios";
function StudentList() {
    const [students, setStudents] = useState([]);
    const [page, setPage] = useState(1);
    const [hasMore, setHasMore] = useState(true);
    const loadStudents = async () => {
        const res = await
        axios.get(`http://localhost:8000/api/students?page=${page}&page_size=3`);
        if (res.data.results.length === 0) {
            setHasMore(false);
        }
    }
}
```

```

    } else {
      setStudents((prev) => [...prev, ...res.data.results]);
    }
  };

  useEffect(() => {
    loadStudents();
  }, [page]);

  const handleScroll = () => {
    if (window.innerHeight + document.documentElement.scrollTop >=
document.documentElement.offsetHeight - 10 && hasMore) {
      setPage((prev) => prev + 1);
    }
  };

  useEffect(() => {
    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll);
  }, [hasMore]);

  return (
    <div>
      <h2>Students</h2>
      <ul>
        {students.map((s) => (
          <li key={s.id}>{s.name}</li>
        ))}
      </ul>
      {!hasMore && <p>No more students</p>}
    </div>
  );
}

export default StudentList;

```

Question 7: GraphQL Integration with Django + React

Expose student data using GraphQL and fetch it in React with Apollo Client.

Input Format (GraphQL Query):

```
query {  
  student(id: 1) {  
    name  
    class  
    marks  
  }  
}
```

Output Format:

```
{  
  "data": {  
    "student": {  
      "name": "Rahul Sharma",  
      "class": "10",  
      "marks": 88  
    }  
  }  
}
```

Example 1

Input:

```
query {  
  student(id: 2) {  
    name  
    attendance  
  }  
}
```

Output:

```
{
  "data": {
    "student": {
      "name": "Anjali Mehta",
      "attendance": 92
    }
  }
}
```

Example 2

Input: Invalid ID

```
query {
  student(id: 999) {
    name
  }
}
```

Output:

```
{
  "errors": [
    {"message": "Student not found"}
  ]
}
```

Answer :

models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    student_class = models.CharField(max_length=10)
    marks = models.IntegerField()
    attendance = models.IntegerField()
```

```
def __str__(self):  
    return self.name
```

schema.py

```
import graphene  
  
from graphene_django import DjangoObjectType  
from .models import Student  
  
class StudentType(DjangoObjectType):  
    class Meta:  
        model = Student  
        fields = ("id", "name", "student_class", "marks", "attendance")  
  
class Query(graphene.ObjectType):  
    student = graphene.Field(StudentType, id=graphene.Int(required=True))  
  
    def resolve_student(root, info, id):  
        try:  
            return Student.objects.get(pk=id)  
        except Student.DoesNotExist:  
            raise Exception("Student not found")  
  
schema = graphene.Schema(query=Query)
```

urls.py

```
from django.urls import path  
  
from graphene_django.views import GraphQLView  
from django.views.decorators.csrf import csrf_exempt  
  
urlpatterns = [  
    path("graphql/", csrf_exempt(GraphQLView.as_view(graphiql=True))),  
]
```

ApolloClientSetup.js

```
import { ApolloClient, InMemoryCache, ApolloProvider } from "@apollo/client";
import React from "react";

const client = new ApolloClient({
  uri: "http://localhost:8000/graphql/",
  cache: new InMemoryCache(),
});

export default function ApolloClientSetup({ children }) {
  return <ApolloProvider client={client}>{children}</ApolloProvider>;
}
```

StudentQuery.js

```
import React from "react";
import { gql, useQuery } from "@apollo/client";

const GET_STUDENT = gql`
  query GetStudent($id: Int!) {
    student(id: $id) {
      name
      studentClass
      marks
      attendance
    }
  }
`;

function StudentQuery({ studentId }) {
  const { loading, error, data } = useQuery(GET_STUDENT, {
    variables: { id: studentId },
  });
}
```

```

if (loading) return <p>Loading...</p>;
if (error) return <p>Error: {error.message}</p>;
return (
  <div>
    <h2>{data.student.name}</h2>
    <p>Class: {data.student.studentClass}</p>
    <p>Marks: {data.student.marks}</p>
    <p>Attendance: {data.student.attendance}%</p>
  </div>
);
}
export default StudentQuery;

```

Question 8: Audit Logging Middleware in Django

Every API call must be logged with user details, request time, and action.

Input Format:

POST /api/update-attendance

```

{
  "user": "teacher_01",
  "student_id": 12,
  "attendance": "Present"
}

```

Output Format (Log Entry in DB):

```

{
  "user": "teacher_01",
  "action": "update-attendance",
  "student_id": 12,
  "timestamp": "2025-09-06T12:30:45Z"
}

```

Example 1

Input:

POST /api/update-marks

```
{  
  "user": "teacher_02",  
  "student_id": 15,  
  "marks": 95  
}
```

Output:

```
{  
  "user": "teacher_02",  
  "action": "update-marks",  
  "student_id": 15,  
  "timestamp": "2025-09-06T14:02:10Z"  
}
```

Example 2

Input:

GET /api/students?class=10

Output:

```
{  
  "user": "student_portal",  
  "action": "view-students",  
  "class": "10",  
  "timestamp": "2025-09-06T15:05:33Z"  
}
```

Answer :

models.py

```
from django.db import models  
  
class AuditLog(models.Model):  
    user = models.CharField(max_length=100)
```

middleware.py

```
import datetime

from .models import AuditLog

class AuditLogMiddleware:

    def __init__(self, get_response):

        self.get_response = get_response

    def __call__(self, request):

        response = self.get_response(request)

        if request.path.startswith("/api/"):

            try:

                user = request.POST.get("user") or request.GET.get("user") or
"anonymous"

                action = request.path.strip("/").replace("api/", "")

                details = {}

                if request.method == "GET":

                    details = request.GET.dict()

                elif request.method in ["POST", "PUT", "PATCH"]:

                    details = request.POST.dict()

                AuditLog.objects.create(

                    user=user,

                    action=action,

                    details=details

                )

            except Exception as e:

                print(e)
```

```
except Exception as e:
    print("Audit log error:", e)
return response
```

settings.py

```
import os
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent
SECRET_KEY = "django-insecure-xyz123"
DEBUG = True
ALLOWED_HOSTS = []
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "school",
]
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
```

```

    "school.middleware.AuditLogMiddleware",
]
ROOT_URLCONF = "project.urls"
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.debug",
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    ],
]
WSGI_APPLICATION = "project.wsgi.application"
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
AUTH_PASSWORD_VALIDATORS = [
    {"NAME":
"django.contrib.auth.password_validation.UserAttributeSimilarityValidator"},

```

```
{
  "NAME": "django.contrib.auth.password_validation.MinimumLengthValidator",
  "NAME": "django.contrib.auth.password_validation.CommonPasswordValidator",
  "NAME": "django.contrib.auth.password_validation.NumericPasswordValidator",
}

LANGUAGE_CODE = "en-us"

TIME_ZONE = "UTC"

USE_I18N = True

USE_TZ = True

STATIC_URL = "static/"

DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"
```

Question 9: Internationalization (i18n) in Django + React

Support **multi-language responses** (English, Hindi) depending on user preference.

Input Format:

GET /api/greeting?lang=hi

Output Format:

```
{
  "message": "नमस्ते, स्वागत है!"
}
```

Example 1

Input:

GET /api/greeting?lang=en

Output:

```
{
  "message": "Hello, welcome!"
}
```

Example 2

Input:

GET /api/greeting?lang=hi

Output:

```
{  
    "message": "नमस्ते, स्वागत है!"  
}
```

Answer :

settings.py

```
LANGUAGE_CODE = "en"  
USE_I18N = True  
LANGUAGES = [  
    ("en", "English"),  
    ("hi", "Hindi"),  
]  
LOCALE_PATHS = [  
    BASE_DIR / "locale",  
]
```

views.py

```
from django.http import JsonResponse  
from django.utils.translation import gettext as _  
def greeting(request):  
    lang = request.GET.get("lang", "en")  
    from django.utils import translation  
    translation.activate(lang)  
    message = _("Hello, welcome!")  
    return JsonResponse({"message": message})
```

urls.py

```
from django.urls import path  
from .views import greeting  
urlpatterns = [  
    path("api/greeting", greeting, name="greeting"),  
]
```

Greeting.js

```
import React, { useState, useEffect } from "react";
import axios from "axios";

function Greeting({ lang }) {
  const [message, setMessage] = useState("");
  useEffect(() => {
    axios.get(`/api/greeting?lang=${lang}`)
      .then(res => setMessage(res.data.message))
      .catch(err => console.error(err));
  }, [lang]);
  return <h2>{message}</h2>;
}

export default Greeting;
```

Question 10: Optimistic UI Updates in React with Django API

When a student's marks are updated, React should **optimistically update UI** before the API confirms.

Input Format (Update Request):

```
{
  "student_id": 20,
  "marks": 90
}
```

Output Format (API Response):

```
{
  "status": "success",
  "student_id": 20,
  "marks": 90
}
```

Example 1

Input:

```
{
```

```
"student_id": 21,  
"marks": 85  
}
```

Output:

```
{  
  "status": "success",  
  "student_id": 21,  
  "marks": 85  
}
```

Example 2

Input (Backend fails but UI already updated):

```
{  
  "student_id": 22,  
  "marks": 95  
}
```

Output:

```
{  
  "status": "error",  
  "message": "Database error: Could not update marks"  
}
```

Answer :

views.py

```
from django.http import JsonResponse  
from django.views.decorators.csrf import csrf_exempt  
from django.utils.decorators import method_decorator  
import json  
  
from .models import Student  
  
@csrf_exempt  
def update_marks(request):
```

```

if request.method == "POST":
    try:
        data = json.loads(request.body)
        student_id = data.get("student_id")
        marks = data.get("marks")
        student = Student.objects.get(id=student_id)
        student.marks = marks
        student.save()
        return JsonResponse({
            "status": "success",
            "student_id": student_id,
            "marks": marks
        })
    except Exception as e:
        return JsonResponse({
            "status": "error",
            "message": f"Database error: {str(e)}"
        }, status=500)

```

urls.py

```

from django.urls import path
from .views import update_marks
urlpatterns = [
    path("api/update-marks", update_marks, name="update_marks"),
]

```

UpdateMarks.js

```

import React, { useState } from "react";
import axios from "axios";

```

```

function UpdateMarks({ studentId, initialMarks }) {
  const [marks, setMarks] = useState(initialMarks);
  const [status, setStatus] = useState("");
  const handleUpdate = async (newMarks) => {
    setMarks(newMarks);
    setStatus("Saving...");
    try {
      const res = await axios.post("/api/update-marks", {
        student_id: studentId,
        marks: newMarks,
      });
      if (res.data.status === "success") {
        setStatus("Saved");
      } else {
        setStatus("Failed ");
      }
    } catch (err) {
      setStatus("Error (UI was updated optimistically)");
    }
  };
  return (
    <div>
      <h3>Student {studentId}</h3>
      <p>Marks: {marks}</p>
      <button onClick={() => handleUpdate(marks + 5)}>+5 Marks</button>
      <p>Status: {status}</p>
    </div>
  );
}
export default UpdateMarks;

```